

Dokumentacja inżynierii wymagań

Maurycy Januszek, Mateusz Rybak, Patryk Pajerski

Listopad 2025

1 Macierz kompetencji zespołu

Kompetencja	Maurycy Januszek	Patryk Pajerski	Mateusz Rybak
Programowanie Python	+	+	+
Obsługa database	+	–	+
Web development	+	+	+
Testowanie oprogramowania	–	–	+
Implementacja modeli uczenia maszynowego	–	–	+
Znajomość docker	–	+	+/-
Znajomość UML	–	+/-	–

Tabela 1: Macierz kompetencji zespołu

2 Zestaw pytań, które zostały sformułowane w celu uszczegółowienia zadanego projektu

Pytanie	Odpowiedź
Jaka jest docelowa platforma sprzętowa systemu?	Laptop (CPU/GPU + kamera)
Jakie dane wejściowe będą przetwarzane?	Kod QR oraz wizerunek twarzy
Jakie informacje mają być raportowane?	Poprawne wejścia (data, godzina, ID pracownika, status wejścia); niepoprawne wejścia (powód odmowy, data, godzina, ID pracownika, zdjęcie z kamry, status wejścia)
Jakie funkcje ma posiadać panel administracyjny?	Dodawanie pracownika, uzupełnianie danych (zdjęcie, informacje pracownika), aktualizacja danych, zarządzane przepustkami, przegląd raportów
Jak długo ma być ważny kod QR?	QR code ważny 3600 minut
Czy system ma generować przepustki?	Tak, generowanie przepustek
Do ilu pracowników ma być dostosowany system?	Do 20 pracowników
Jaki jest maksymalny dopuszczalny czas przetwarzania?	3 sekundy
Jaki jest wymagany minimalny poziom skuteczności?	Min. 90%
Jak długo przechowywane są raporty?	6 miesięcy

Tabela 2: Zestawienie pytań i odpowiedzi dotyczących wymagań systemu

3 Ustalony format danych wejściowych

Poniższa tabela definiuje strukturę danych przesyłanych z terminala wejściowego do serwera w celu weryfikacji tożsamości pracownika. Dane są formatowane jako obiekt JSON, aby umożliwić łatwe przesyłanie złożonych informacji, takich jak obraz.

Nazwa Pola	Typ Danych	Opis
qr_code	string	Unikalny identyfikator pracownika
face_image	string (Base64)	Obraz twarzy po konwersji do formatu Base64

Tabela 3: Struktura danych wejściowych wysyłanych z terminala.

Przykładowy format (JSON):

```
1 {  
2   "qr_code": "7asfda1283addfda...",  
3   "face_image": "iVBORw0KGgoAAAAN..."  
4 }
```

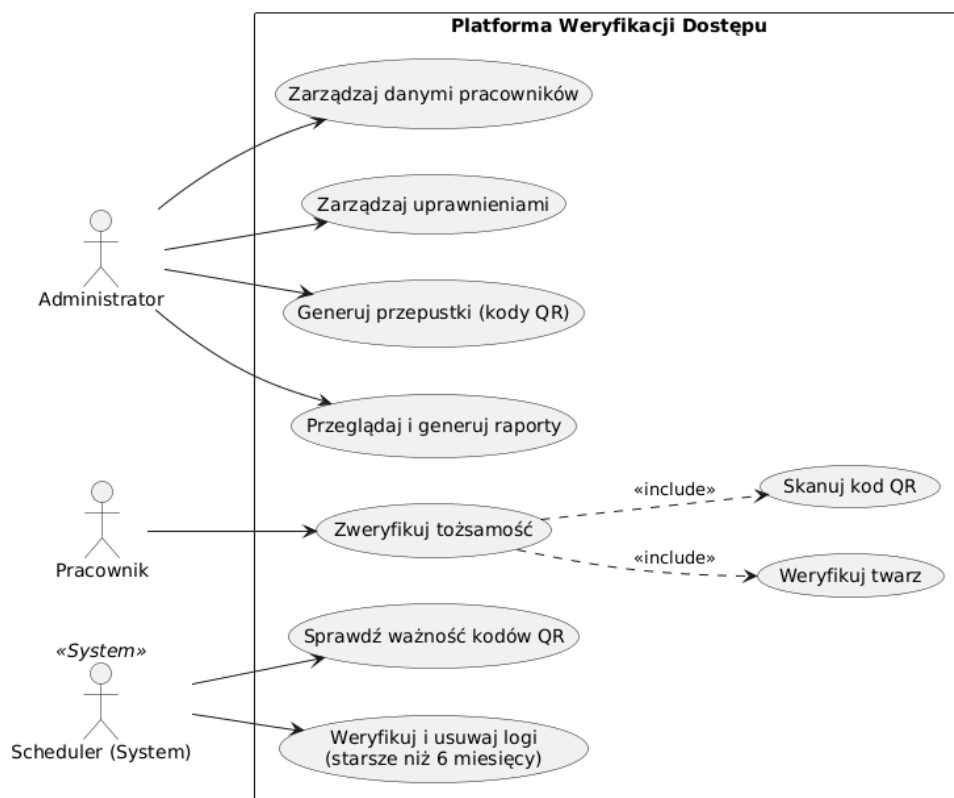
4 Modelowanie systemu za pomocą tabeli

Aktorzy	Użytkownik, Administrator
Opis	Celem będzie weryfikacja tożsamości użytkownika w celu zaraportowania jego obecności na terenie obiektu
Dane	Unikalny ograniczony czasowo kod QR oraz skan twarzy użytkownika
Wyzwalacz	Po otrzymaniu niezbędnych danych system na ich podstawie feryfikuje tożsamość użytkownika
Odpowiedź	Autoryzacja lub jej dorzucenie oraz zarejestrowanie wejścia/poinformowanie o próbie naruszenia
Uwagi	System pobiera od użytkownika dane, kod QR oraz skan twarzy i sprawdza czy kod QR nie wygasł oraz czy jest poprawny. Dopasowywyuje do kodu QR skan twarzy użytkownika oraz zapisuje zdjęcie w celu uaktualnienia danych.

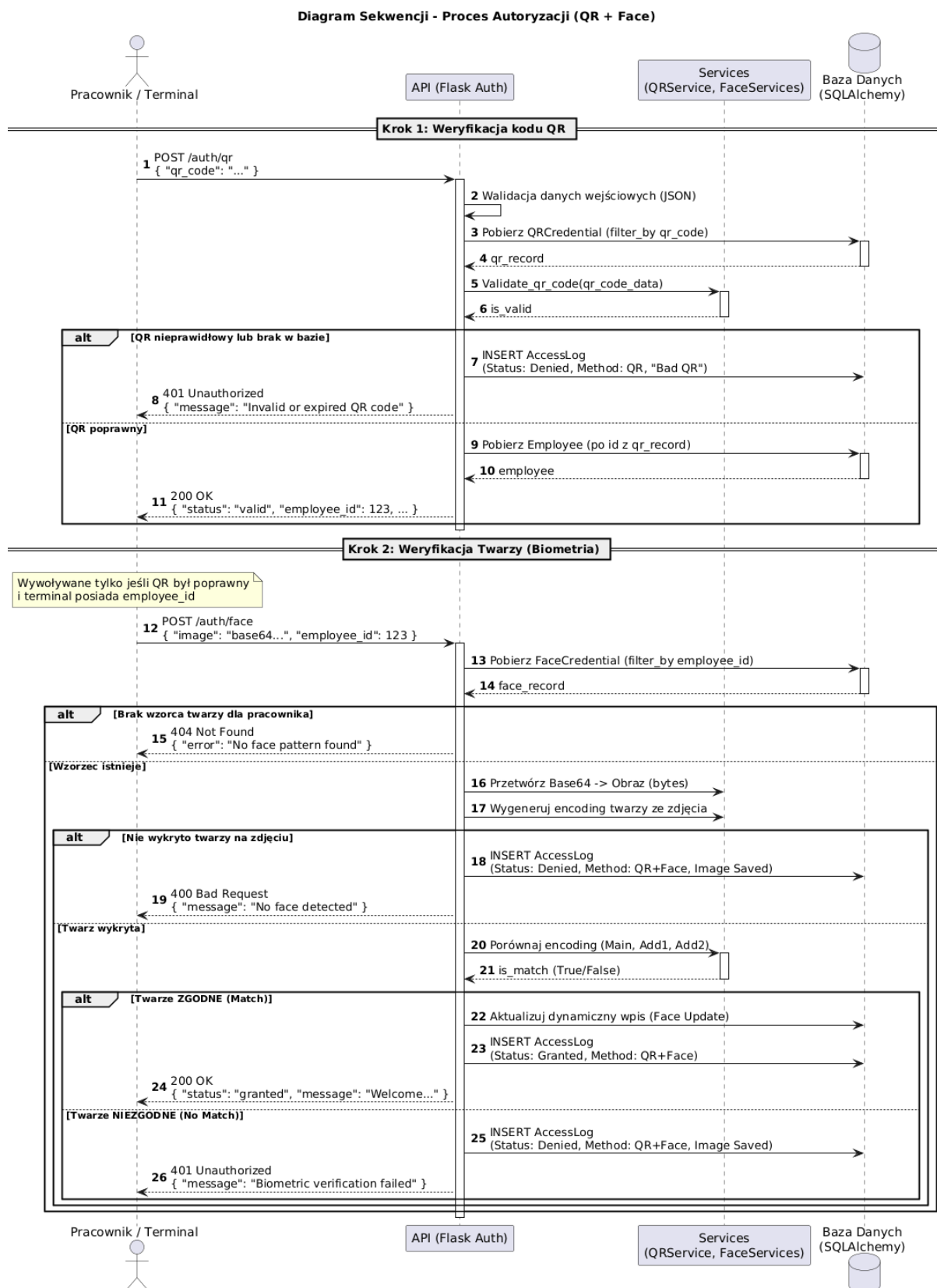
Tabela 4: Model systemu

5 Przedstawienie systemu za pomocą diagramu UML

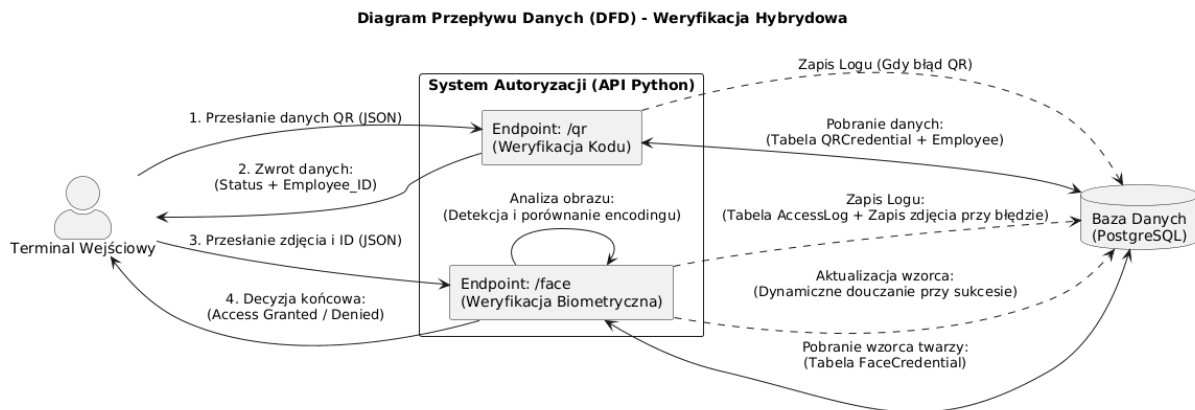
Poniżej przedstawiono diagramy dla Platformy Weryfikacji Dostępu.



Rysunek 1: Diagram przypadków użycia systemu.

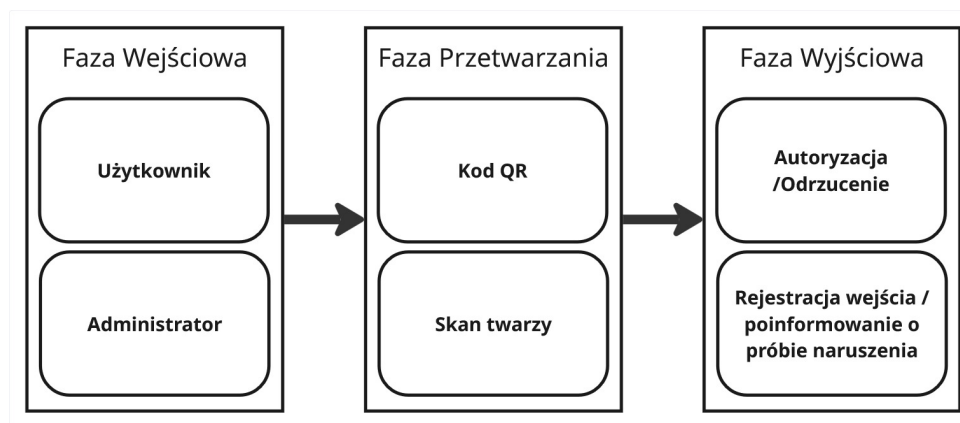


Rysunek 2: Diagram przepływu danych.



Rysunek 3: Diagram sekcacyjny.

6 Projekt architektury opracowanego systemu



Rysunek 4: Projekt architektury opracowanego systemu

Każdy z trzech dużych prostokątów reprezentuje osobny komponent systemu. Zadaniem systemu jest najpierw odebranie danych wejściowych od użytkownika, następnie przetworzenie ich w celu weryfikacji tożsamości, a na końcu zapisanie informacji o poprawnym lub niepoprawnym wejściu.

Pierwszym i najważniejszym elementem systemu jest pobranie poprawnych danych, czyli ważnego kodu QR oraz aktualnego skanu twarzy użytkownika. Dzięki temu możliwe jest dalsze, bezbłędne działanie programu.

Drugim kluczowym komponentem systemu jest moduł przetwarzający, który odpowiada za weryfikację tożsamości. System sprawdza poprawność kodu QR, jego czas ważności oraz dopasowuje skan twarzy do danych zapisanych w bazie. Algorytm musi działać szybko i niezawodnie, aby zapewnić przetwarzanie danych w czasie nie dłuższym niż 3 sekundy.

Ostatni komponent odpowiada za generowanie odpowiedzi końcowej, czyli autoryzację lub odrzucenie dostępu do obiektu. W przypadku poprawnego dopasowania system rejestruje wejście użytkownika wraz z datą, godziną i identyfikatorem pracownika. W przypadku nieudanej weryfikacji system zapisuje próbę naruszenia oraz powód odrzucenia. Dodatkowo przechowywane są raporty z działania systemu przez okres 6 miesięcy, co pozwala administratorowi na kontrolę bezpieczeństwa i bieżącą analizę.

Całość działania systemu wspierana jest przez panel administracyjny, którego zadaniem jest zarządzanie pracownikami, uaktualnianie danych biometrycznych oraz przegląd raportów. System jest zoptymalizowany do obsługi maksymalnie 20 pracowników, co umożliwia sprawne działanie i wysoką skuteczność procesów identyfikacyjnych.

7 Sugerowany język implementacji: Python i Javascript

Po analizie wymagań funkcjonalnych i нефункциональных, w szczególności potrzeby integracji z modelami uczenia maszynowego, zalekomendowano wybór następującego stosu technologicznego opartego na architekturze klient-serwer:

- **Warstwa Backendu (logika serwerowa): Python**
- **Warstwa Frontendu (interfejsy użytkownika): JavaScript z nowoczesnym frameworkiem (np. React)**

7.1 Backend: Python

Python został wybrany do implementacji logiki serwerowej, obsługi bazy danych oraz przetwarzania danych biometrycznych z następujących powodów:

- **Niezrównane wsparcie dla uczenia maszynowego:** Python jest de facto standardem w dziedzinie AI i przetwarzania obrazów. Dostępność dojrzałych bibliotek, takich jak **OpenCV**, **Dlib**, **TensorFlow** czy **PyTorch**, drastycznie przyspiesza i upraszcza implementację kluczowej funkcjonalności systemu – rozpoznawania twarzy.
- **Dojrzałe frameworki webowe:** Framework **Flask** pozwala na szybkie budowanie bezpiecznych, wydajnych i skalowalnych interfejsów API (**REST API**), które będą służyć do komunikacji z warstwą frontendową. Zapewnia również solidne wsparcie dla operacji na bazach danych.
- **Szybkość rozwoju i czytelność kodu:** Czysta i zwięzła składnia Pythona pozwala na szybsze tworzenie oprogramowania oraz ułatwia jego późniejsze utrzymanie i rozwój przez zespół.
- **Ogromny ekosystem:** Python posiada ogromną bazę gotowych pakietów do niemal każdego zadania (np. generowanie raportów PDF, obsługa kolejek zadań), co minimalizuje potrzebę implementowania standardowych rozwiązań od zera.

7.2 Frontend: JavaScript

JavaScript jest naturalnym wyborem dla wszystkich komponentów systemu, z którymi w interakcję wchodzi użytkownik (panel administratora oraz oprogramowanie na terminalach wejściowych).

- **Uniwersalny język interfejsów webowych:** Jest to jedyna technologia natywnie wspierana przez wszystkie przeglądarki internetowe, co czyni ją idealnym wyborem do stworzenia panelu administracyjnego dostępnego z dowolnego komputera w sieci firmowej.
- **Nowoczesne frameworki:** Wykorzystanie frameworka **React** umożliwia budowę szybkich, interaktywnych i responsywnych interfejsów użytkownika. Ułatwia zarządzanie stanem aplikacji, co jest kluczowe w przypadku panelu z dynamicznie aktualizowanymi raportami i listami danych.
- **Zastosowanie na terminalach wejściowych:** Oprogramowanie terminala również może być zrealizowane jako aplikacja webowa działająca w trybie pełnoekranowym (kiosk). JavaScript, poprzez API przeglądarek, ma dostęp do kamery (**getUserMedia API**) oraz możliwość obsługi skanerów kodów QR, co pozwala na unifikację technologii w całym projekcie.

7.3 Uzasadnienie architektury klient-serwer

Podział na backend w Pythonie i frontend w JavaScriptcie przynosi następujące korzyści:

- **Rozdzielenie logiki od prezentacji:** Zmiany w interfejsie użytkownika nie wpływają na logikę biznesową i odwrotnie. Ułatwia to testowanie i równoległą pracę deweloperów.
- **Skalowalność:** Obie warstwy mogą być skalowane niezależnie. W razie wzrostu obciążenia można zwiększyć zasoby serwera aplikacyjnego, nie modyfikując oprogramowania na terminalach.
- **Elastyczność:** Jasno zdefiniowane API pozwala w przyszłości na łatwe dołączanie nowych klientów (np. aplikacji mobilnej dla ochrony) bez ingerencji w logikę backendu.